

DEEPPFAKE DETECTION SYSTEM

Image, Audio & Video Forensic Analysis Platform

Final Year Project — Complete Development Guide

From Dataset Collection to Deployment, with Automated
Forensic Reporting for Cybercrime Complaints

A Step-by-Step Technical & Project Guide

Table of Contents

1. Project Overview & Objectives
2. System Architecture
3. Technology Stack
4. Phase 1 — Planning & Requirement Analysis
5. Phase 2 — Dataset Collection (Image, Audio, Video)
6. Phase 3 — Data Preprocessing
7. Phase 4 — Model Selection & Training
8. Phase 5 — Model Evaluation
9. Phase 6 — Backend API Development
10. Phase 7 — Frontend Website Development
11. Phase 8 — Forensic Report Generation Module
12. Phase 9 — Integration & Testing
13. Phase 10 — Deployment
14. Suggested Project Timeline (12–16 Weeks)
15. Legal, Ethical & Chain-of-Custody Considerations
16. Evaluation Metrics for Your Report/Viva
17. Common Pitfalls & How to Avoid Them
18. Reference Datasets, Papers & Tools

1. Project Overview & Objectives

This guide walks you through building a full-stack **Deepfake Detection Platform** that accepts an image, audio clip, or video uploaded by a user (your client), runs it through machine learning models trained to detect AI-generated or manipulated media, and produces a downloadable **forensic analysis report**. That report is designed to give the client the evidence summary they need when filing a complaint with a cybercrime cell or police station.

Core Objectives

- Detect manipulated/synthetic **images** (face-swap, GAN-generated, diffusion-generated).
- Detect manipulated/synthetic **audio** (voice cloning, TTS-generated speech).
- Detect manipulated **video** (face-swap deepfakes, lip-sync manipulation, temporal artifacts).
- Provide a confidence score and visual/technical explanation (heatmaps, frame-level scores, spectrogram evidence).
- Auto-generate a signed, timestamped **PDF forensic report** suitable for attaching to a police/cybercrime complaint.
- Deploy the system as a usable website with authentication, upload, processing queue, and report history.

What This Guide Covers

The guide is organized as ten build phases, from planning to deployment, followed by a suggested timeline, evaluation guidance for your viva/defense, and a reference list of datasets and papers you can cite in your report.

2. System Architecture

A production-style deepfake detector is best built as a decoupled system: a web frontend, a backend API, an asynchronous inference/worker layer (since video/audio processing is slow), a database, and a report generator. This separation also makes your final-year project look and behave like a real product during evaluation.

Layer	Responsibility	Suggested Technology
Frontend (Client)	Upload UI, drag-and-drop, result dashboard, report download, login	React.js / Next.js + Tailwind CSS
Backend API	Auth, file upload handling, job queueing, serving results	FastAPI (Python) or Node.js + Express
Inference Workers	Runs the ML models on image/audio/video, returns scores	Python (PyTorch/TensorFlow) + Celery/RQ + Redis
Database	Users, uploads, job status, model outputs, report metadata	PostgreSQL / MongoDB
File/Object Storage	Stores uploaded media & generated reports securely	AWS S3 / local disk with encryption for dev
Report Generator	Builds the forensic PDF report from model output	Python (ReportLab / WeasyPrint)

Deployment	Hosting, containerization, HTTPS, scaling	Docker + Nginx + AWS/GCP/Render/Railway
------------	---	--

High-Level Data Flow

- 1 Client registers/logs in and uploads an image, audio, or video file on the website.
- 2 Backend validates the file (type, size, format) and stores it in object storage.
- 3 Backend creates a background job and pushes it to a queue (Redis/Celery) — this keeps the website responsive while heavy models run.
- 4 A worker picks up the job, runs the appropriate model (image/audio/video pipeline), and generates a probability score plus supporting evidence (heatmap, frame scores, spectrogram, etc.).
- 5 Results are saved to the database; the frontend polls or receives a websocket/notification when the job completes.
- 6 User views the result dashboard: verdict (Likely Real / Likely Fake), confidence score, and visual explanation.
- 7 User clicks 'Generate Report' — the backend compiles a structured PDF forensic report with case reference number, hash of the original file (for chain-of-custody), timestamps, model version, and findings.
- 8 User downloads the report to attach to a cybercrime complaint (e.g., on India's cybercrime.gov.in portal, or their local police cyber cell, depending on jurisdiction).

Note: Your website itself does not file the police complaint — it produces the technical evidence report the client can submit. Keep this distinction clear in your documentation; it also avoids legal complications around your project acting as a legal authority.

3. Technology Stack

Recommended Stack (Beginner-to-Intermediate Friendly)

Component	Recommended Choice	Why
ML Framework	PyTorch	Most deepfake research code/pretrained models are released in PyTorch
Image Model	XceptionNet / EfficientNet-B4	Proven backbones for FaceForensics++ style detection
Video Model	CNN + LSTM, or frame-wise Xception + temporal aggregation	Captures both spatial artifacts and temporal inconsistency
Audio Model	RawNet2 / LCNN with spectrogram features	Standard architectures in the ASVspoof anti-spoofing literature
Face Detection/Crop	MTCNN or RetinaFace (via facenet-pytorch)	Needed to crop faces before feeding image/video models
Backend	FastAPI	Async-friendly, auto docs, integrates well with Python ML code
Frontend	React.js / Next.js	Component-based, easy dashboard + charts (recharts) for results
Queue	Celery + Redis	Handles slow video/audio inference without blocking the API
Database	PostgreSQL	Reliable relational storage for users, jobs, reports
Report PDF	ReportLab (Python)	Full control over layout for a professional forensic report
Explainability	Grad-CAM / Grad-CAM++	Produces heatmaps showing which image regions triggered detection

Note: You can substitute TensorFlow/Keras throughout if that's what your coursework uses — the architecture and workflow stay the same.

4. Phase 1 — Planning & Requirement Analysis (Week 1)

Before writing code, define scope clearly — this is what examiners look for in a final-year project report.

- **Scope document:** which media types (image/audio/video), what manipulation types you'll detect (face-swap, lip-sync, voice cloning, GAN faces, diffusion-generated images), and explicit exclusions.
- **User roles:** Guest (limited free scan) vs Registered user (full report + history).
- **Non-functional requirements:** max file size, supported formats (jpg/png, mp3/wav, mp4/mov), expected processing time, target accuracy.
- **Literature review:** read 8–10 papers on deepfake detection (see Section 18) and summarize approaches in your report — this is usually mandatory for final-year submissions.
- **Draw the system architecture diagram and ER diagram** for your database — include these in your project report.

5. Phase 2 — Dataset Collection (Weeks 2–3)

You do not need to create your own deepfakes — use established, citation-ready public research datasets. Below are the standard datasets used in academic deepfake detection work, organized by media type.

5.1 Image / Video Deepfake Datasets

Dataset	Content	Access
FaceForensics++ (FF++)	1000 real + manipulated videos (DeepFakes, Face2Face, FaceSwap, NeuralTextures)	Request via github.com/ondyari/FaceForensics — academic email required
Celeb-DF (v2)	590 real + 5,639 high-quality celebrity deepfake videos	Request form: github.com/yuezunli/celeb-deepfakeforensics
DFDC (Deepfake Detection Challenge)	~128,000 videos, released by Meta/AWS/Microsoft for Kaggle challenge	Kaggle: 'Deepfake Detection Challenge'
WildDeepfake	Real-world deepfake videos scraped from the internet (more diverse/harder)	GitHub: deepfakeinthewild
140k Real and Fake Faces	GAN-generated (StyleGAN) vs real face images — great for the image-only pipeline	Kaggle dataset, open access
DFFD (Diverse Fake Face Dataset)	Covers GAN synthesis, face-swap, and expression manipulation	cvlab.cse.msu.edu (request access)

5.2 Audio Deepfake Datasets

Dataset	Content	Access
ASVspoof 2019 / 2021	Bonafide vs spoofed (TTS/voice-conversion) speech — the standard anti-spoofing benchmark	asvspoof.org — free registration
WaveFake	Generated audio from 6 different neural vocoders (MelGAN, WaveGlow, etc.)	Zenodo, open access
In-The-Wild Audio Deepfake Dataset	Real-world cloned celebrity/politician speech clips	Available via research paper repo (deepfake-total.com)
FoR (Fake or Real) Dataset	Large TTS-generated vs human speech dataset	Kaggle / Bil-Kent research page

Note: Most of these datasets require you to fill an academic-use request form with your institution email — start these requests in Week 1 since approval can take a few days.

5.3 If You Cannot Get Full Dataset Access in Time

- Kaggle-hosted subsets (140k Real/Fake Faces, DFDC sample set, FoR dataset) are instantly downloadable with no request process — good fallback for a working prototype.
- You can also use **pretrained models** released alongside these datasets (e.g., FF++ pretrained Xception weights) and fine-tune only on a smaller subset you can access, which is realistic and acceptable for a final-year project.

- Document clearly in your report which portion of the dataset you personally trained on vs which pretrained weights you used — examiners respect transparency here.

6. Phase 3 — Data Preprocessing (Week 4)

Image/Video Pipeline

- 1 Extract frames from videos (e.g., 1 frame every 0.5s using OpenCV/ffmpeg).
- 2 Run face detection (MTCNN/RetinaFace) and crop+align each face to a fixed size (e.g., 224x224).
- 3 Balance classes: equal-ish numbers of real vs fake samples per dataset to avoid biased training.
- 4 Apply data augmentation: random crop, horizontal flip, JPEG compression artifacts, brightness/contrast jitter — this mimics the real-world image quality your client's uploads will have.
- 5 Split into train/validation/test (e.g., 70/15/15), split by **source video/identity** not by frame, so the same face doesn't leak across train and test sets.

Audio Pipeline

- 1 Resample all clips to a consistent sample rate (16kHz is standard for ASVspoof-style models).
- 2 Convert waveform to features: Mel-spectrogram, MFCCs, or use raw waveform for RawNet2.
- 3 Trim silence, normalize volume, and pad/crop to a fixed duration window (e.g., 4 seconds) for batch training.
- 4 Split train/val/test by speaker where possible, to test generalization to unseen voices.

7. Phase 4 — Model Selection & Training (Weeks 5–8)

7.1 Image Detector

Start from an **ImageNet-pretrained** backbone (Xception or EfficientNet-B4) and fine-tune the final layers on your face-crop dataset with a binary classification head (real=0, fake=1). Use binary cross-entropy loss, Adam optimizer (lr ~1e-4), and early stopping on validation loss.

7.2 Video Detector

Two common approaches, pick one based on time available:

- **Frame-based (simpler, recommended for final-year timeline):** run the image detector on multiple sampled frames and average/vote the per-frame scores into one video-level score.
- **Sequence-based (more advanced):** feed a sequence of frame embeddings (from a CNN) into an LSTM or a small Transformer to model temporal inconsistency across frames — more accurate but heavier to train.

7.3 Audio Detector

Train a CNN over spectrograms (LCNN architecture from the ASVspoof baseline) or RawNet2 on raw waveform, again as a binary classifier (bonafide vs spoofed). Use the ASVspoof official baseline training script as your starting point rather than writing an anti-spoofing pipeline from scratch — this is standard practice and acceptable to cite.

7.4 Training Infrastructure

- Use **Google Colab Pro** or your college GPU lab for training — full training of these models on a laptop CPU is impractical.
- Log training with TensorBoard or Weights & Biases so you have accuracy/loss curves ready for your report.

- Save checkpoints regularly (every epoch) — export the best model as a .pt or .h5 file.
- Typical training time: image model ~2–4 hrs, audio model ~1–3 hrs, video (frame-based) reuses the image model. Budget 2–3 weeks including experimentation, not just the raw training time.

8. Phase 5 — Model Evaluation (Week 9)

Evaluate each model on its held-out test set using standard metrics, and keep the numbers — they go directly into your final report and defense slides.

Metric	What It Tells You
Accuracy	Overall correct predictions — can be misleading if classes are imbalanced
Precision / Recall	How many flagged fakes are truly fake (precision) vs how many real fakes were caught (recall)
F1-Score	Balance between precision and recall — good single summary number
AUC-ROC	How well the model separates real vs fake across all thresholds
EER (Equal Error Rate)	Standard metric in the audio anti-spoofing literature — lower is better
Confusion Matrix	Visual breakdown of false positives/negatives — include this in your report

Note: Also test each model on a small set of samples you generate yourself (e.g., a free online face-swap tool, or a TTS voice clone) that were NOT in the training data at all — this demonstrates real-world generalization to your examiners, which is more convincing than in-dataset test accuracy alone.

9. Phase 6 — Backend API Development (Weeks 9–10)

Core API Endpoints

Endpoint	Method	Purpose
/api/auth/register, /login	POST	User registration & JWT-based login
/api/upload	POST	Accepts image/audio/video file, stores it, creates a job
/api/jobs/{id}/status	GET	Poll job status: queued / processing / done / failed
/api/jobs/{id}/result	GET	Returns verdict, confidence score, evidence data (heatmap URL, frame scores)
/api/jobs/{id}/report	GET	Generates/returns the downloadable PDF forensic report
/api/history	GET	Lists a user's past scans and reports

Backend Implementation Notes

- Load each trained model **once** at worker startup (not per-request) to avoid reloading overhead.
- Use Celery workers (separate process from the API) for inference so video/audio processing doesn't block user requests.
- Store the SHA-256 hash of every uploaded file at upload time — this hash goes into the forensic report and helps prove the analyzed file hasn't been altered (chain of custody).
- Set strict file size/type limits and scan uploads before processing (basic security hygiene, mention this in your report).
- Rate-limit uploads per user to prevent abuse of your GPU inference resources.

10. Phase 7 — Frontend Website Development (Weeks 10–12)

Key Pages

- **Landing page:** explains what the tool does, disclaimers about limitations, and a call-to-action to upload.
- **Upload page:** drag-and-drop for image/audio/video, file-type validation, progress bar while processing.
- **Result dashboard:** verdict badge (Likely Authentic / Likely Manipulated), confidence percentage, visual evidence — e.g., Grad-CAM heatmap overlay for images/video frames, waveform + spectrogram highlight for audio.
- **Report page:** preview of the generated forensic report with a 'Download PDF' button.
- **History/dashboard page:** list of past scans for logged-in users.
- **About/How it works page:** explains your models and datasets at a high level — useful during your viva demo too.

UX/Trust Considerations

- Always show a confidence score/range, never just a hard 'FAKE' label — deepfake detectors are not 100% certain, and your UI should reflect that honestly.
- Add a visible disclaimer: 'This tool provides an automated technical assessment. For legal proceedings, verification by a certified forensic expert is recommended.' This protects the credibility and honesty of your project.

11. Phase 8 — Forensic Report Generation Module (Week 12–13)

This is the feature that makes your project stand out — an auto-generated PDF report formatted like a technical evidence document, which the client attaches when filing a cybercrime complaint (e.g., on India's National Cyber Crime Reporting Portal, or their local police cyber cell — jurisdictions vary, so mention this generically in your UI).

Recommended Report Sections

- 1 **Header:** Report title, unique case/report ID, generation timestamp, your platform name/logo.
- 2 **Submitted File Details:** original filename, file type, file size, SHA-256 hash, upload timestamp.
- 3 **Analysis Summary:** media type analyzed, model(s) used with version numbers, overall verdict, confidence score.
- 4 **Detailed Findings:** for images/video — heatmap of manipulated regions, per-frame confidence graph; for audio — spectrogram with flagged segments, per-segment confidence.
- 5 **Methodology Note:** one paragraph describing the detection approach in plain language (for a non-technical police officer to understand).
- 6 **Limitations Disclaimer:** clearly state this is an automated technical assessment, not a certified forensic/legal opinion, and false positives/negatives are possible.
- 7 **Digital Signature/Hash of Report:** hash the generated report itself so it can't be silently edited after generation — mention this as a chain-of-custody feature in your documentation.

Build this with **ReportLab** in Python (same library used to generate this guide) — you get full control over layout, tables, and embedded images/charts, which is ideal for a professional-looking evidentiary document.

12. Phase 9 — Integration & Testing (Week 13–14)

- **Unit tests:** test preprocessing functions, API endpoints, and report generation independently.
- **Integration tests:** full flow — upload → job processing → result → report download.
- **Load/stress testing:** confirm the queue handles multiple simultaneous uploads without crashing (use a tool like Locust).
- **User Acceptance Testing:** have 5–10 classmates/friends try the site with sample real and fake files, collect feedback on clarity of results.
- **Cross-browser/mobile testing** of the frontend.

13. Phase 10 — Deployment (Week 15)

- 1 Containerize each service (frontend, backend, worker) with Docker; use docker-compose for local orchestration.
- 2 Push Docker images and deploy to a cloud platform — Railway, Render, or an AWS EC2/GPU instance for the inference workers if you need GPU at inference time.
- 3 Set up HTTPS via Let's Encrypt/Nginx reverse proxy.
- 4 Configure environment variables/secrets (DB credentials, JWT secret) properly — never commit them to GitHub.
- 5 Set up basic monitoring/logging (e.g., simple request logging + error alerts) so you can debug issues during your live demo.
- 6 If GPU hosting is too costly for a student project, run inference on CPU with a lighter model variant (e.g., EfficientNet-B0) and mention this trade-off explicitly in your report as a scalability consideration.

14. Suggested Project Timeline (12–16 Weeks)

Week(s)	Milestone
1	Requirement analysis, architecture design, dataset access requests submitted
2–3	Dataset collection & organization (image, audio, video)
4	Preprocessing pipelines for all three media types
5–8	Model training & experimentation (image, audio, video)
9	Model evaluation, metric reporting, error analysis
9–10	Backend API development (auth, upload, job queue)
10–12	Frontend website development
12–13	Forensic PDF report module
13–14	Integration testing, bug fixing, UX polish
15	Deployment to a live/staging server
16	Documentation, final report writing, presentation/viva prep

15. Legal, Ethical & Chain-of-Custody Considerations

- Clearly state in your report and website UI that your tool provides an **automated technical opinion**, not a certified legal/forensic determination — actual cybercrime cases typically require verification by government forensic labs (e.g., in India, a Central Forensic Science Laboratory).
- Preserve the **original uploaded file** unmodified and record its hash at upload time — this supports the chain-of-custody story in your report.
- Handle uploaded media as sensitive personal data: encrypt storage, restrict access, and add a data retention/deletion policy — mention GDPR/India's DPDP Act principles in your report if applicable to your institution's expectations.
- Add a Terms of Use clarifying the platform must not be used to generate deepfakes, only to detect them, and that misuse is the user's responsibility.

- If demoing with real people's faces/voices, use only consenting volunteers or public research datasets — never someone's likeness without permission.

16. Evaluation Metrics for Your Report/Viva

Examiners typically want to see these clearly presented, ideally as charts in your final report/slides:

- Per-model accuracy, precision, recall, F1, AUC-ROC on held-out test data.
- Confusion matrices for each model.
- A comparison table if you tried more than one architecture (e.g., Xception vs EfficientNet).
- Qualitative examples: a few real and fake samples with the model's predicted score and Grad-CAM heatmap, side-by-side.
- System-level metrics: average end-to-end processing time per media type, to demonstrate practicality.

17. Common Pitfalls & How to Avoid Them

- **Data leakage:** splitting frames from the same video across train/test inflates accuracy artificially — always split by video/identity, not by frame.
- **Overfitting to one dataset:** a model trained only on FaceForensics++ often fails on real-world deepfakes; test on WildDeepfake or your own samples too.
- **Ignoring class imbalance:** real-world uploads are mostly real content — track false positive rate carefully, not just accuracy.
- **Treating output as binary certainty:** always present a confidence score with context, not just 'FAKE'/'REAL'.
- **Underestimating video processing time:** frame-by-frame inference on long videos can be slow — sample frames intelligently (e.g., 1 fps) rather than processing every frame.
- **Skipping the disclaimer/legal framing:** examiners and real users will ask about legal validity — address it proactively in your UI and report.

18. Reference Datasets, Papers & Tools

Datasets

- FaceForensics++ — Rossler et al., 2019 — github.com/ondyari/FaceForensics
- Celeb-DF (v2) — Li et al., 2020 — github.com/yuezunli/celeb-deepfakeforensics
- DFDC — Dolhansky et al., 2020 (Meta/AWS/Microsoft) — Kaggle
- WildDeepfake — Zi et al., 2020
- ASVspoof 2019/2021 — asvspoof.org
- WaveFake — Frank & Schönherr, 2021 — Zenodo

Foundational Papers (Good for Your Literature Review)

- 'FaceForensics++: Learning to Detect Manipulated Facial Images' (Rossler et al., ICCV 2019)
- 'The Deepfake Detection Challenge (DFDC) Dataset' (Dolhansky et al., 2020)
- 'Xception: Deep Learning with Depthwise Separable Convolutions' (Chollet, 2017) — the backbone architecture
- 'ASVspoof 2019: A Large-Scale Public Database of Synthesized, Converted and Replayed Speech' (Todisco et al., 2019)
- 'RawNet2: End-to-End Anti-Spoofing with Raw Waveform' (Tak et al., 2021)
- 'Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization' (Selvaraju et al., 2017)

Useful Libraries

- facenet-pytorch — face detection/cropping (MTCNN)
- PyTorch / torchvision — model training
- librosa — audio feature extraction
- OpenCV / ffmpeg-python — video frame extraction
- ReportLab — PDF report generation
- FastAPI, Celery, Redis, PostgreSQL — backend infrastructure

This guide is intended as a technical and planning reference for a final-year academic project. Model accuracy claims, legal procedures, and complaint-filing processes vary by institution and jurisdiction — verify current cybercrime reporting procedures for your region before finalizing the client-facing parts of your report/UI.